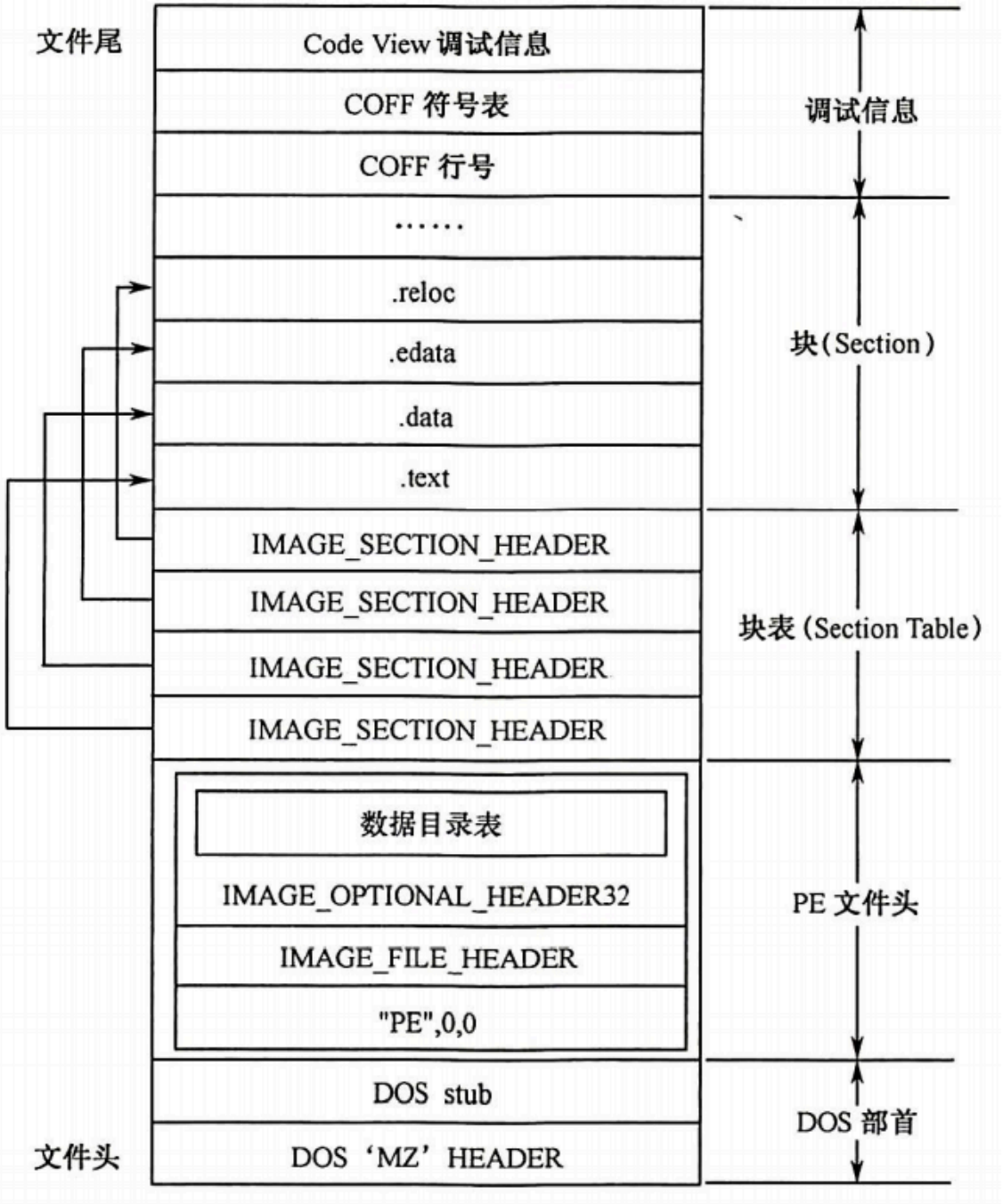
逆向-PE文件结构

原创 编码的凯文 于 2021-05-21 09:15:45 发布 阅读量1.1k 收藏 3 点赞数 CC 4.0 BY-SA版权

分类专栏: 409 安全

首先复习一下PE文件的结构：


文件尾
Code View 调试信息
COFF 符号表
COFF 行号
……
.reloc
.edata
.data
.text
IMAGE_SECTION_HEADER
IMAGE_SECTION_HEADER
IMAGE_SECTION_HEADER
IMAGE_SECTION_HEADER
数据目录表
IMAGE_OPTIONAL_HEADER32
IMAGE_FILE_HEADER
"PE",0,0
DOS stub
文件头
DOS ‘MZ’ HEADER
调试信息
块(Section)
块表(Section Table)
PE 文件头
DOS 部首

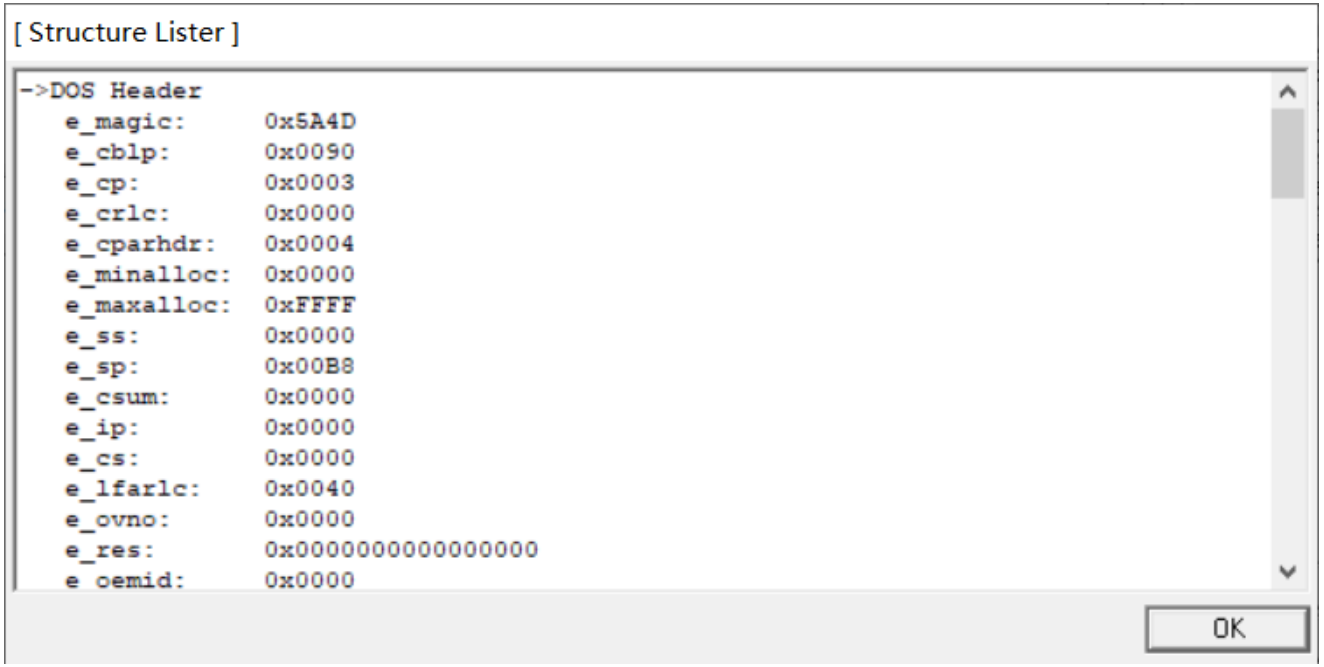
MS-DOS头

DOS部首有MZ Header和Dos stub两个部分组成，这两部分通常合成为Dos 文件头，其中Dos stub多由编译器自动生成，用户较少关注，PE文件的第一个字节为IMAGE_DOS_HEADER,其结构如下：

```
IMAGE_DOS_HEADER_STRUCT{
  +0h    e_magic      WORD  ?            ;DOS 可执行文件标记 “MZ”
  +2h    e_cblp       WORD  ?
  +4h    e_cp         WORD  ?
  +6h    e_crlc       WORD  ?
  +8h    e_cparhdr    WORD  ?
  +0ah   e_minalloc   WORD  ?
  +0ch   e_maxalloc   WORD  ?
  +0eh   e_ss         WORD  ?
  +10h   e_sp         WORD  ?
  +12h   e_csum       WORD  ?
  +14h   e_ip         WORD  ?            ;DOS 代码入口 IP
  +16h   e_cs         WORD  ?            ;DOS 代码入口 CS
  +18h   e_lfarlc     WORD  ?
  +1ah   e_ovno       WORD  ?
  +1ch   e_res        WORD  4 dup(?)
  +24h   e_oemid      WORD  ?
  +26h   e_oeminfo    WORD  ?
  +28h   e_res2       WORD 10 dup(?)
  +3ch   e_lfanew     DWORD ?            ;指向 PE 文件头"PE",0,0
} IMAGE_DOS_HEADER_ENDS
```

其中e_magic和e_lfanew两个字段比较重要，e_magic被称为魔术字，所有能与MS-DOS兼容的可执行文件内，该值都被设置为5A4DH，对应ASCII码为“MZ”，是DOS系统创建者之一 Mark Zbikowski 的缩写。e_lfanew指向PE文件头部，是真正的PE文件头的相对偏移（RAV），占用4字节。

使用lordPE查看文件



PE文件头

PE Header（PE 头部）是 PE 相关结构 IMAGE_NT_HEADERS 的简称，其中包含了许多 PE 装载器用到的重要域。执行体在执行时，装载器从IMAGE_DOS_HEADER结构中的e_lfanew字段找到PE Header的起始偏移量，用其加上基址得到PE文件头的指针。

`pNTHeader = dosHeader + dosHeader->e_lfanew`

IMAGE_NT_HEADERS由三个字段组成（左边数字为到PE文件头的偏移量），结构如下：

```
IMAGE_NT_HEADERS STRUCT
  +0h    Signature            DWORD                    ?        ;PE 文件标识
  +4h    FileHeader           IMAGE_FILE_HEADER        <>
  +18h   OptionalHeader       IMAGE_OPTIONAL_HEADER32  <>
IMAGE_NT_HEADERS ENDS
```

Singture字段：有效PE文件中，该值被设置为0x00004550,ASCII为PE00，标志着PE文件头的开始，e_lfanew就是指向这个字段。

IMAGE_FILE_HEADER:包含了PE文件的一些基本信息，其中一个域指出了IMAGE_OPTIONAL_HEADER的大小，其结构如下：

```
IMAGE_FILE_HEADER STRUCT
  +04h Machine                  WORD    ?    ;运行平台
  +06h NumberOfSections         WORD    ?    ;文件的区块数
  +08h TimeDateStamp            DWORD   ?    ;文件创建日期和时间
  +0Ch PointerToSymbolTable     DWORD   ?    ;指向符号表（用于调试）
  +10h NumberOfSymbols          DWORD   ?    ;符号表中符号的个数（用于调试）
  +14h SizeOfOptionalHeader     WORD    ?    ;IMAGE_OPTIONAL_HEADER32 结构的大小
  +16h Characteristics          WORD    ?    ;文件属性
IMAGE_FILE_HEADER ENDS
```

名称	含义
Machine	运行所要求的 CPU。对于 Intel 平台，该值是 14Ch。
NumberOfSections	文件的节数目，文件中增加或删除一个节，就需要修改这个值。
TimeDateStamp	文件创建日期和时间。
PointerToSymbolTable	用于调试。
NumberOfSymbols	用于调试。
SizeOfOptionalHeader	指示 OptionalHeader 结构的大小，必须为有效值。
Characteristics	关于文件信息的标记，比如文件是 exe 还是 dll。

当文件为exe时characteristics为34，当文件为DLL时其值为8450。

OptionalHeader （可选头部） 是 PE Header 的第三个成员， 是IMAGE_OPTIONAL_HEADER 结构体，它由 224 个字节组成，虽然名为“可选头部”但它是“必需”的，因为IMAGE_FILE_HEADER不足以完全定义PE文件属性，因此该可选文件中定义了更多数据，两者联合构成了完整的PE文件头结构，其成员比较复杂，它包含了 PE 文件的逻辑分布信息，该结构共有 31 个域，主要成员表如下。

名称	含义
AddressOfEntryPoint	PE 装载器准备运行的 PE 文件的第一个指令的 RVA。
ImageBase	PE 文件的优先装载地址。
SizeOfImage	内存中整个 PE 映像体的尺寸。
SizeOfHeaders	头+节表的大小，也就等于文件尺寸减去文件中所有节的尺寸。
Subsystem	NT 用来识别 PE 文件属于哪个子系统。
SizeOfStackReserve	线程初始堆栈的保留大小。
SizeOfStackCommit	一开始即被提交（committed）给线程初始堆栈的内存数量。
SizeOfHeapReserve	保留给最初的 process heap 的虚拟内存数量。
SizeOfHeapCommit	一开始即被提交（committed）给 process heap 的内存数量。
DataDirectory	IMAGE_DATA_DIRECTORY 结构数组。

IMAGE_OPTIONAL_HEADER 中最重要的一个域是 DataDirectory，它是一个包含 16 个元素的结构数组，每一个结构对应着一个 Section（注意这里的 Section 是按照按作用进行划分的 Section，不是最终生成的 PE 文件中包含的块），结构中的两个域分别描述了该 Section 的 RVA 和 SIZE; 这样一来加载器就能够通过这个数组迅速在 image 中找到特定的 Section，后面讲到的输入表、输出表都要用到这个数组中相应的元素。

区块

区块表在PE文件头和原始数据之间，其中包含每个块在映像中的信息，分别指向不同区块实体。
Section Table （区块表）在PE文件中是紧挨着PE Header的，是Image_Section_Table 的结构数组的简称，其成员个数由 IMAGE_FILE_HEADER结构中的 NumberOfSections 域值决定。
其结构定义如下：

```
IMAGE_SECTION_HEADER   STRUCT
   Name                          BYTE8 DUP (?)     ;8 字节的块名
   union Misc                                      ;区块尺寸
     PhysicalAddress             DWORD  ?
     VirtualSize                 DWORD  ?
   Ends
   VirtualAddress                DWORD  ?          ;区块的 RVA 地址
   SizeOfRawData                 DWORD  ?          ;在文件中对齐后的尺寸
   PointerToRawData              DWORD  ?          ;在文件中的偏移
   PointerToRelocations          DWORD  ?          ;在 OBJ 文件中使用，重定位的偏移
   PointerToLinenumbers          DWORD  ?          ;行号表的偏移（供调试用）
   NumberOfRelocations           WORD   ?          ;在 OBJ 文件中使用，重定位项数目
   NumberOfLinenumbers           WORD   ?          ;行号表中行号的数目
   Characteristics               DWORD  ?          ;区块的属性
IMAGE SECTION HEADER  ENDS
```

名称	含义
Name	块的名称。
VirtualAddress	本块的 RVA（相对虚拟地址）。
SizeOfRawData	经过文件对齐处理后节尺寸，PE 装载器提取本域值了解需映射入内存的块的字节数。
PointerToRawData	这是块基于文件的偏移量，PE 装载器通过本域值找到块数据在文件中的位置。
Characteristics	包含标记以指示块属性。

区块是PE文件的真正内容划分，每一块是拥有共同属性的数据的集合，块的种类和含义如下图：

块名	含义
.arch	最初的构建信息(Alpha Architecture Information)
.bss	未经初始化的数据
.CRT	C 运行期只读数据
.data	已经初始化的数据
.debug	调试信息
.didata	延迟输入文件名表
.edata	导出文件名表
.idata	导入文件名表
.pdata	异常信息(Exception Information)
.rdata	只读的初始化数据
.reloc	重定位表信息
.rsrc	资源
.text	.exe 或.dll 文件的可执行代码
.tls	线程的本地存储器
.xdata	异常处理表

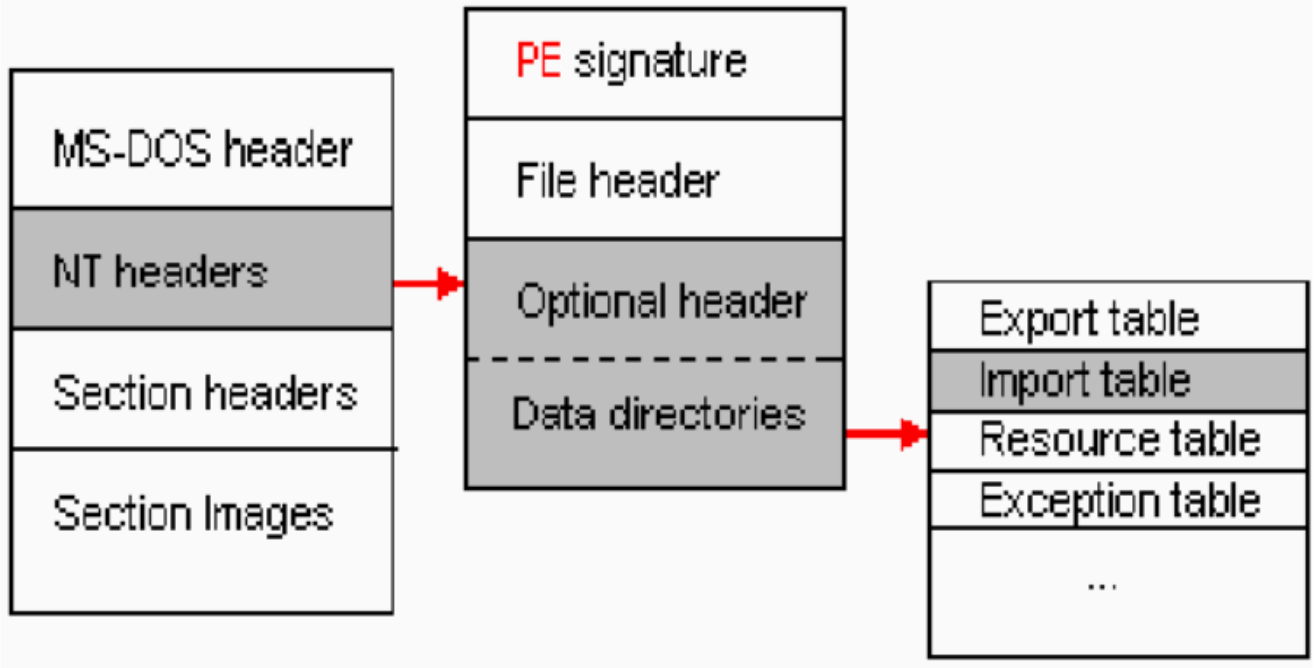
块在映像中按起始地址（RAV）排列，而非块名，所以块名在使用中无关紧要。

名　称	描　述
.text	默认的代码区块，它的内容全是指令代码。PE 文件运行在 32 位方式下，不受 16 位段的约束，所以没有理由把代码放到不同的区块中。链接器把所有目标文件的 .text 块链接成一个大的 .text 块。如果使用 Borland C++，其编译器将产生的代码存储于名为 code 的区域，其链接器链接的结果是使代码块的名称不是 .text，而是 code
.data	默认的读/写数据区块。全局变量、静态变量一般放在这里
.rdata	默认的只读数据区块，但程序很少用到该块中的数据。至少有两种情况要用到 .rdata 块。一是在 Microsoft 链接器产生的 EXE 文件中，用于存放调试目录；二是用于存放说明字符串。如果程序的 DEF 文件中指定了 DESCRIPTION，字符串就会出现在 .rdata 块中
.idata	包含其他外来 DLL 的函数及数据信息，即输入表。将 .idata 区块合并到另一个区块已成为惯例，典型的是 .rdata 区块。链接器默认仅在创建一个 Release 模式的可执行文件时才将 .idata 区块合并到另一个区块中
.edata	输出表。当创建一个输出 API 或数据的可执行文件时，链接器会创建一个 .EXP 文件，这个 .EXP 文件包含一个 .edata 区块，它会被加入最后的可执行文件中。与 .idata 区块一样，.edata 区块也经常被发现合并到了 .text 或 .tdata 区块中

.rsrc	资源。包含模块的全部资源，例如图标、菜单、位图等。这个区块是只读的，无论如何都不应该命名为 .rsrc 以外的名字，也不能被合并到其他区块里
.bss	未初始化数据。很少使用，取而代之的是执行文件的 .data 区块的 VirtualSize 被扩展到足够大以存放未初始化的数据
.crt	用于支持 C++ 运行时（CRT）所添加的数据
.tls	TLS 的意思是线程局部存储器，用于支持通过 __declspec（thread）声明的线程局部存储变量的数据，既包括数据的初始化值，也包括运行时所需的额外变量
.reloc	可执行文件的基址重定位。基址重定位一般只是 DLL 需要，而不是 EXE 需要。在 Release 模式下，链接器不会给 EXE 文件加上基址重定位，重定位可以在链接时通过 /FIXED 开关关闭
.sdata	相对于全局指针的可被定位的“短的”读/写数据，用于 IA-64 和其他使用一个全局指针寄存器的体系结构。IA-64 上的常规大小的全局变量放在这个区块里
.srdata	相对于全局指针的可被定位的“短的”只读数据，用于 IA-64 和其他使用一个全局指针寄存器的体系结构
.pdata	异常表，包含一个 CPU 特定的 IMAGE_RUNTIME_FUNCTION_ENTRY 结构数组，DataDirectory 中的 IMAGE_DIRECTORY_ENTRY_EXCEPTION 指向它。它用于异常处理，是基于表的体系结构，就像 IA-64。唯一不使用基于表的异常处理的架构体系是 x86
.debug$S	OBJ 文件中 Codeview 格式的符号。这是一个变量长度的 Codeview 格式的符号记录流
.debug$T	OBJ 文件中 Codeview 格式的类型记录。这是一个变量长度的 Codeview 格式的类型记录流
.debug$P	当使用预编译的头时，可以在 OBJ 文件中找到它
.drectve	包含链接器命令，只能在 OBJ 中找到它。命令是能被传递给链接器命令行的字符串，例如“-defaultlib:LIBC”。命令用空格字符分开
.didat	延迟载入的输入数据，只能在非 Release 模式的可执行文件中找到。在 Release 模式下，延迟载入的数据会被合并到另一个区块中

区块的合并和对齐：编译器会自动生成一些了标准区块，但程序员也可以通过

`#pragma data_seg("xx")`
来将所有C++处理的数据放到xx数据块内，而不是默认的data数据块。可以通过链接器把两个属性相同的区块合并，合并区块的优点是节约磁盘和内存空间，如下将text和rdata合并为text区块。
`/MERGE:.rdata=.text`
区块的大小需要对齐，一种是磁盘内的，一种是内存中的，两个值在PE文件头中指出，FlieAlignment定义了磁盘的对齐值，SectionAlignment定义了内存的对齐值。
#输入表
可执行文件使用其他DLL的代码或数据称为输入。
载入PE文件时，定位所有被输入的函数与数据，并让被载入文件使用那些地址，这个过程需要输入表(import table)来完成，表中保存函数名和驻留DLL等动态链接信息。当程序调用 DLL 中的函数时，编译器编译后的 CALL 指令并不会把控制权直接传给 DLL 中的函数，而是传给一个 JMP DWORD PTR [XXXXXXXX] 指令，后者位于.text 中，JMP 指令跳转到一个地址，此地址储存在.idata 的一个 DWORD之中。
如果加载器要调用 DLL 中的函数，首先必须知道所加载的程序调用了哪些DLL 以及 DLL 中的哪些函数，然后找出这些函数的地址，并把这些地址添入到.idata 的 DWORD 中。输入表（Import Table）的作用就是告诉加载器，程序调用了哪些 DLL 的哪些函数，并且通过 Export Table 找出这些函数的地址。
输入表在PE文件中的位置如图：



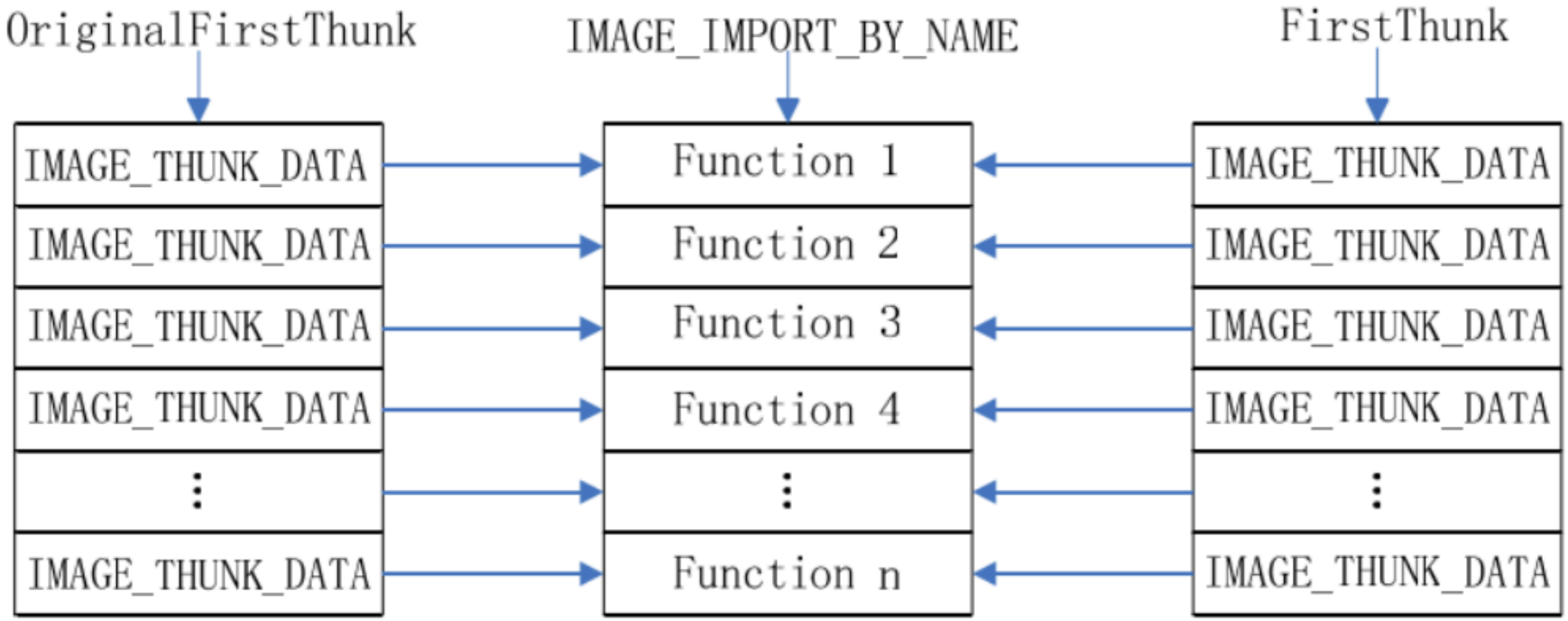
输入表是一个 IMAGE_IMPORT_DESCRIPTOR (IID)结构数组，每个结构包含 PE文件引入函数的一个相关 DLL 的信息,最后有一个内容全为0的IID作为结尾，

IID的结构如下：

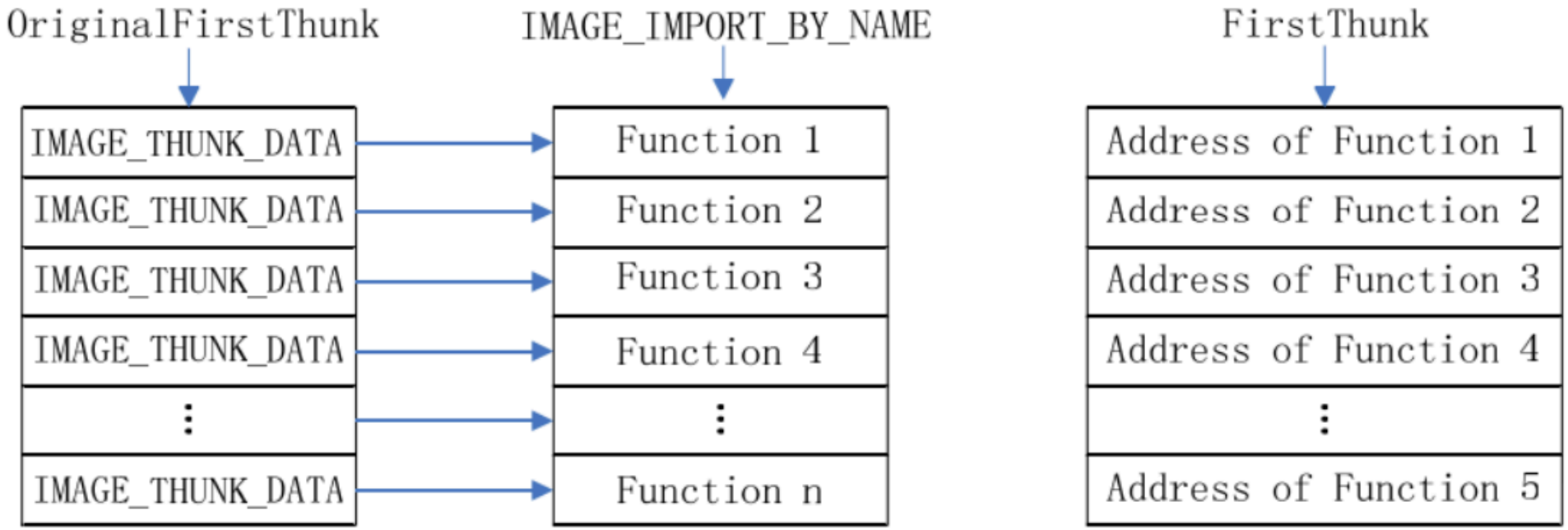
```
IMAGE_IMPORT_DESCRIPTOR STRUCT
    union                                ;00h
         Characteristics       DWORD ?
         OriginalFirstThunk    DWORD ?
    ends
    TimeDateStamp              DWORD ? ;04h
    ForwarderChain             DWORD ? ;08h
    Name                       DWORD ? ;0Ch
    FirstThunk                 DWORD ? ;10h
IMAGE_IMPORT_DESCRIPTOR ENDS
```

MAGE_IMPORT_DESCRIPTOR 结构体的第一项是一个 union。该成员是指向 输 入 名 称 表 （ Import Name Table， 简 称 INT ） 数 组 的 RVA 。 INT 是 IMAGE_THUNK_DATA 结构的数组，每个 IMAGE_THUNK_DATA 对应于一个输入函数，这个 DWORD（即 IMAGE_THUNK_DATA union）的内容由文件是否被加载（加载前后内容不同）、函数是以名称输入还是以序号输入而定（输入方式不 同 其 内 容 也 不 同 ） 。 当 一 个 函 数 以 序 号 输 入， EXE 文 件 的 IMAGE_THUNK_DATA DWORD 中的最高位（0x80000000）设立。如果函数以名 称 输 入， IMAGE_THUNK_DATA DWORD 就 内 含 一 个 RVA， 指 向 IMAGE_IMPORT_BY_NAME 结 构， 由 于 一 个 输 入 函 数 对 应 一 个IMAGE_THUNK_DATA union， 而 当 函 数 以 名 称 输 入 的 时 候， IMAGE_THUNK_DATA 又指向一个 IMAGE_IMPORT_BY_NAME 结构，所以此时 一 个 输 入 函 数 对 应 一 个 IMAGE_IMPORT_BY_NAME 结 构 。

#输入地址表



载入器一一检阅每一个 IMAGE_THUNK_DATA ，然后进一步通过其他结构找出引入函数的地址。然后用引入函数真实地址来替代由 FirstThunk 指向的 IMAGE_THUNK_DATA 数组里的元素值。因此当 PE 文件准备执行时，上图已转换成下图，这也证实了前面所说的 IMAGE_THUNK_DATA 结构的内容不仅和函数是按序号或者名字引入有关，而且和文件是否加载有关。由于此时 FirstThunk指 向 的IMAGE_THUNK_DATA 数 组 已 经 被 修 改 ， 所 以 它 和 IMAGE_IMPORT_BY_NAME 数组的关系也被切断。



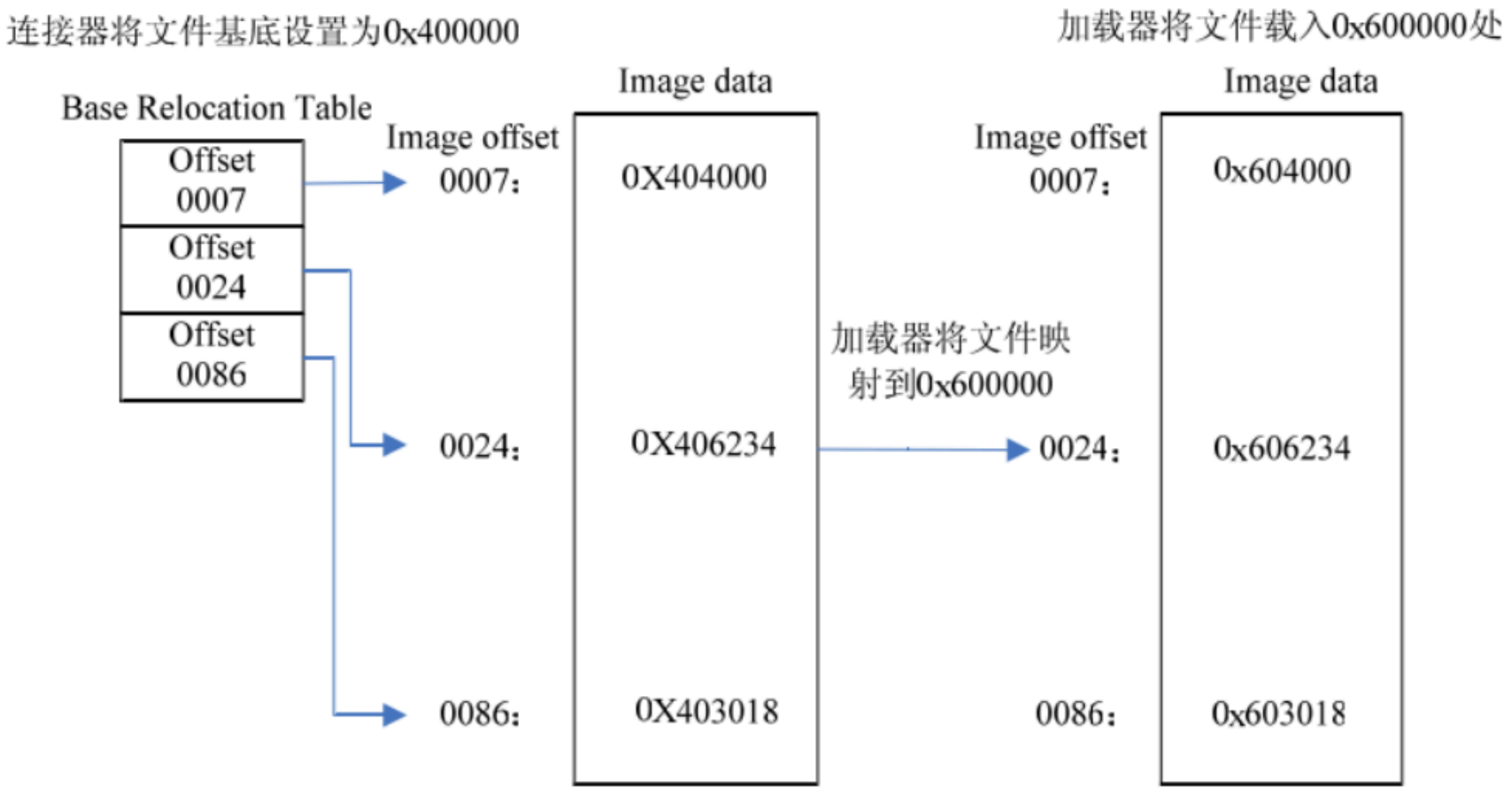
#输出表

创建DLL时实际上创建了能被其他DLL或EXE调用的函数，装载文件根据该输出修改被执行文件的导入地址表(IAT)，DLL文件通过输出表向系统提供函数名，序号和入口地址等信息。

#基址重定位

PE 文件中的每个模块都有一个优先加载地址（ImageBase，也可以称作基地址），其值由连接器给出，连接器生成指令中的地址是相对于优先加载地址而言的，如果模块加载的起始地址不是优先加载地址，那么程序中的指令地址就需要修改，重定位表往往单独作为一块，用.reloc表示。 **（Q：那不初始化基地址的值即可？）**

其原理图如下：


连接器将文件基底设置为0x400000
加载器将文件载入0x600000处
Base Relocation Table
Offset 0007
Offset 0024
Offset 0086
Image data
Image offset
0007：
0X404000
0024：
0X406234
0086：
0X403018
加载器将文件映射到0x600000
Image data
Image offset
0007：
0x604000
0024：
0x606234
0086：
0x603018